PROJECT REPORT

Crack Detection Using OpenCV

This project uses OpenCV image processing to detect visible cracks in road and concrete surfaces for easier damage inspection.

Section C - Team 21

Team Members - Names and Registration Numbers

Team Member Name	Registration No.
Pragyan Garg	20233212
Pushpraj Singh	20233226
Nishan Patgiri	20233204
Piyush Rai	20233210

Crack Detection Using OpenCV

Image-based surface crack detection through OpenCV processing

Abstract

Highway, road and tunnel maintenance requires early detection of surface cracks. Repair costs can be reduced when cracks are identified before they worsen. This project uses images of roads and concrete surfaces and applies image-processing techniques to extract visible crack information. Image capture quality is important: clear, high-resolution images help the later processing and detection steps. The approach can support visual inspections made with image-capturing devices mounted on vehicles or other platforms. Its output highlights crack-related features to assist further examination.

Introduction

Road safety depends in part on the condition of asphalt and concrete surfaces. Cracks and holes can develop over time, and regularly inspecting long stretches of highway manually is difficult. An automatic visual detection method can help identify damage so that road authorities can plan maintenance earlier.

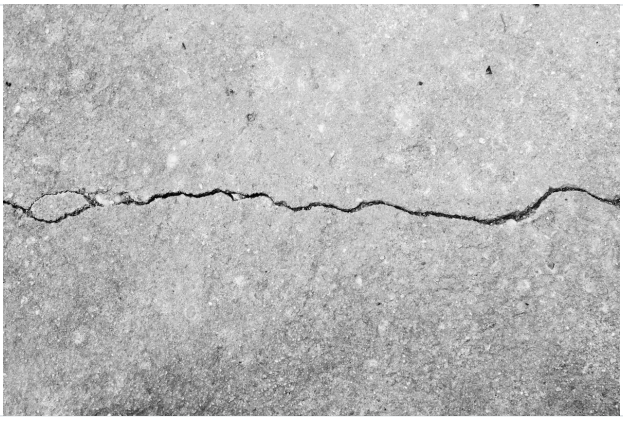
As described in the README, images of road surfaces are collected first and then analyzed to locate visible cracks. Different crack types may require different repairs. The goal of the project is to detect cracks automatically from captured images using OpenCV.

Methodology

The README presents four stages: 1. image capture, 2. image processing, 3. image segmentation and 4. feature extraction. The following figures show the two example crack images used throughout its processing demonstration.



Input image 1 — Cracked_01.jpg



Input image 2 — Cracked_07.jpg

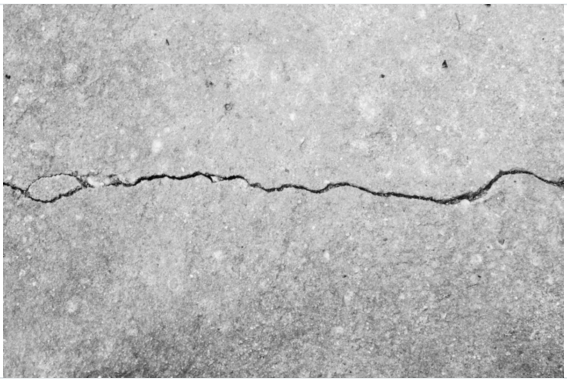
Methodology: Image Processing

Grayscale conversion and averaging

The original image is converted to grayscale, followed by a 3 × 3 averaging blur. This produces a smoother image for the subsequent enhancement and segmentation operations.



Blurred image — sample 1



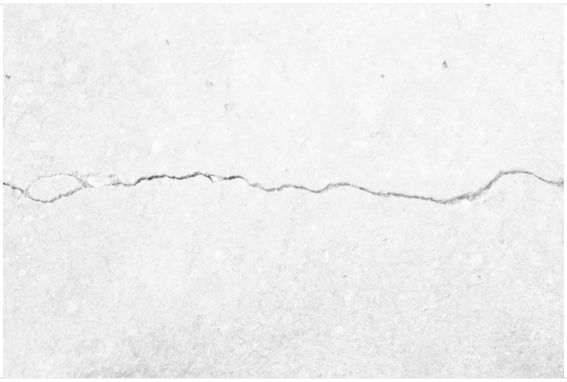
Blurred image — sample 7

Logarithmic transformation

A logarithmic transformation adjusts pixel intensities to enhance darker image details. The script scales the transformed values into an 8-bit image before further processing.



Log-transformed image — sample 1



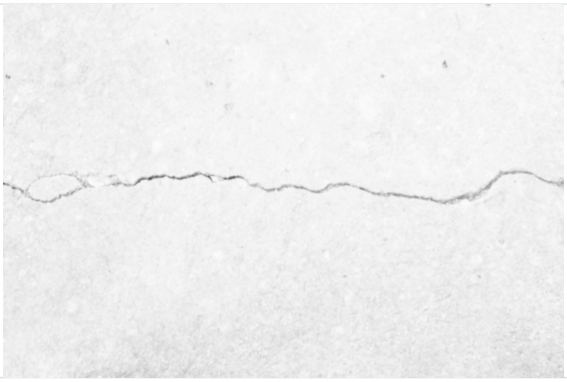
Log-transformed image — sample 7

Bilateral filtering

The bilateral filter smooths the image while limiting blur across strong intensity boundaries. This helps preserve crack edges. The implementation uses a neighborhood diameter of 5 and sigma values of 75.



Bilateral-filtered image — sample 1



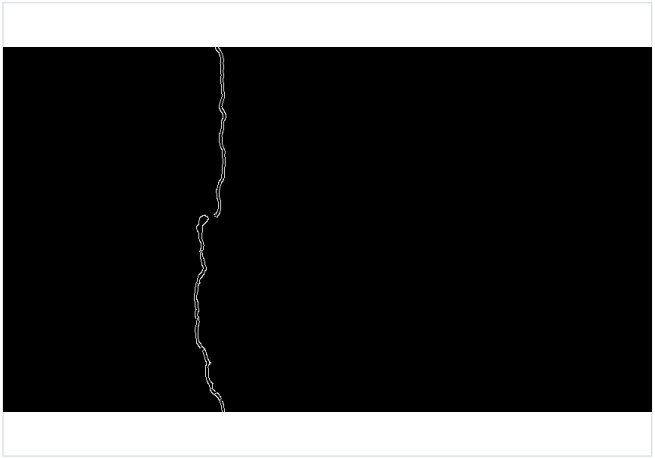
Bilateral-filtered image — sample 7

Methodology: Image Segmentation

Canny edge detection

Canny edge detection extracts structural information from the filtered image. As explained in the README, it calculates intensity gradients, applies non-maximum suppression to thin the edges, and uses double-threshold hysteresis to reduce false edge responses.

In the Python implementation, cv2.Canny is applied to the bilateral-filtered image with thresholds of 100 and 200. The figures below show the detected edges.



Canny edges — sample 1

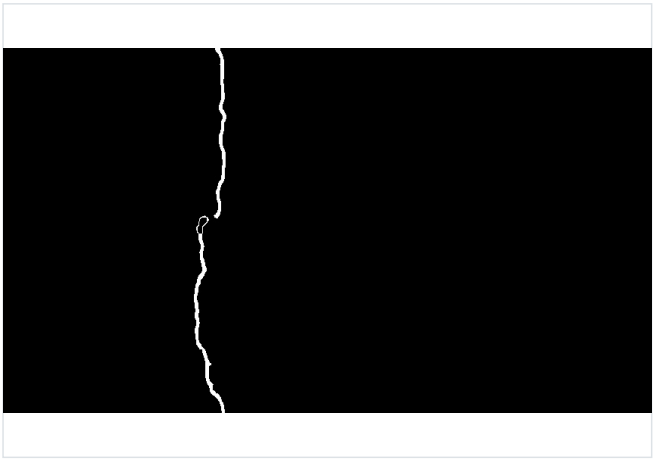


Canny edges — sample 7

Morphological closing

Morphological operations work on image shapes using a structuring element, or kernel. Closing consists of dilation followed by erosion. It joins small gaps so that the main crack line becomes more continuous and visible.

The code applies cv2.MORPH_CLOSE to the Canny edges with a 5 × 5 kernel of ones. The resulting images are shown below.



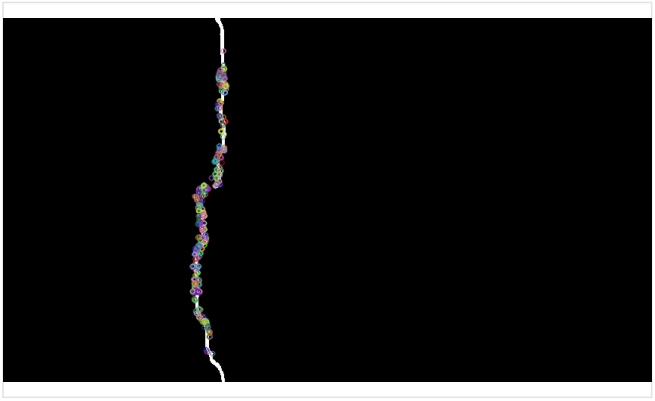
Morphological closing — sample 1



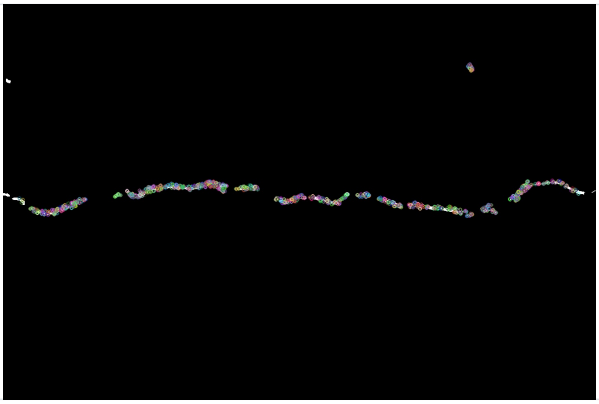
Morphological closing — sample 7

Methodology: Feature Extraction

The README discusses SIFT, SURF and ORB as feature-extraction methods and uses ORB (Oriented FAST and Rotated BRIEF) for this project. The Python script creates an ORB detector with 1,500 requested features, detects keypoints in the closed-edge image and draws them on the output. The colored markers reveal features along the visible crack boundaries.



Feature-detected image — sample 1



Feature-detected image — sample 7

Results and Discussion

The repository README states that the approach was tried on approximately 20 crack and non-crack images. It reports that cracks were clearly visible in many outputs and suggests 80–90% accuracy may be possible with clear input images. This is an author-reported estimate, not a documented benchmark with a confusion matrix or independent test metrics. The output images highlight crack features; classification into crack types would require an additional method.

Conclusion

This OpenCV-based pipeline demonstrates a simple process for enhancing images, isolating crack edges and displaying features with ORB. Image clarity and surface conditions can affect the results. Further work could evaluate the method on a larger labeled dataset and add crack-type classification.

References

1. M. Ann and P. Johnson, Best Practices Handbook on Asphalt Pavement Maintenance. Minnesota T2 Center / LTAP Program, 2000.
2. T. D. Walker, Pavement Surface Evaluation and Rating (PASER) Manuals. Wisconsin Transportation Information Center, 2002.
3. R. K. Kay, Pavement Surface Condition - Rating Manual. Washington State Department of Transportation, 1992.
4. L. Li, L. Sun, G. Ning and S. Tan, “Automatic pavement crack recognition based on BP neural network,” PROMET, 26(1), 11–22, 2014.
5. NHTSA, National Motor Vehicle Crash Causation Survey: Report to Congress, 2008.
6. J. M. Palomares, J. González and E. Ros, “Detección de bordes en imágenes con sombras mediante LIP–Canny,” AERFAI, 2005.
7. J. S. Miller, R. B. Rogers and G. R. Rada, Distress Identification Manual for the Long-Term Pavement Performance Project, Appendix A, 1993.